Lab 4: Obstacle Avoidance, Mapping, and Path Planning

Group 6
Abdalrahman Nasser - abdnas-2@student.ltu.se
Addo Dominic - domadd-2@student.ltu.se
Aittokallio Deborah - debait-2@student.ltu.se
Bardia Baigy - barbai-1@student.ltu.se

Objective

The objective of this lab was to implement a full navigation stack for the TurtleBot3, moving from reactive obstacle avoidance to map-based deliberation. Specifically, we aimed to:

1. Implement **Potential Fields** from the course material for real-time collision avoidance.
2. Generate an **Occupancy Grid Map** using LIDAR data from manual teleoperation.
3. Apply a path planning algorithm (**A***) to navigate the static map autonomously.

Methodology

1. Obstacle Avoidance (Potential Fields)

We implemented a reactive controller using the Artificial Potential Fields method. The robot's commanded velocity became the result of combining an attractive force towards the goal with repulsive forces away from nearby obstacles. The controller runs at each control cycle, reads the latest LIDAR scan, and computes a virtual force vector in the robot frame that is then converted to linear and angular velocity commands.

- **Attractive Force:** Modeled as a proportional vector directed toward the goal position in the robot frame. The vector size grows with the distance to the goal up to a certain limit, preventing the robot from accelerating indefinitely on long straight segments. When the robot is close to the goal (within a set radius), the attractive term is set to zero, avoiding oscillations around the final pose.
- **Repulsive Force:** Generated by LIDAR points within a critical safety distance.

For each LIDAR ray that detects an obstacle closer than the chosen critical safety distance, we compute a repulsive vector pointing away from the obstacle, with magnitude inversely proportional to the measured distance (closer obstacles produce stronger repulsion). All active repulsive vectors are then summed to form a single repulsive force that pushes the robot away from walls and corners. Doing so ensures that obstacles directly in front of the robot dominate the response, while far obstacles have a negligible effect.

- **Combination and Control Command:** The final control vector is obtained by summing the attractive and repulsive components. This resultant vector defines a dynamic "virtual goal" for the robot to chase. We fed this target into a Proportional (P) controller like that developed in Lab 2, which computes the angular velocity to align with the vector and modulates linear velocity based on the vector's magnitude. This approach allowed us to reuse our tested path-tracking logic while effectively treating the potential field as a high-level trajectory generator.
- **Tuning:** The key challenge was balancing the attractive and repulsive gains of our Potential Fields controller. If the attractive gain was too high, the robot tended to “cut corners” and skim dangerously close to walls, while if the repulsive gain was too high, the robot slowed down excessively or started to oscillate in narrow passages. We experimentally tuned the repulsive force to ensure the robot would veer off early enough without oscillating violently in narrow corridors. This was done by reducing local minima in corners, allowing for smoother turns through the maze.

2. Environment Mapping

We manually teleoperated the robot through a maze we built, capturing its intricacies as a point cloud we would use for the path-planning task. To do so, we recorded the /scan topic ROS bag while driving slowly to minimize odometry drift and motion blur in the LIDAR frames. The raw mapping data proved unusable for planning due to significant noise, so we applied a custom cleaning pipeline where we did the following:

- **Tight Cropping:** We discarded all points outside the coordinate box [0.0, 0.5] to [2.2, -2.2] to focus strictly on the navigation corridor.
- **Outlier Rejection:** We applied a radius-based filter to remove stray "ghost" pixels caused by sensor noise.
- **Grid Downsampling:** We snapped points to a 0.5 cm sub-grid to thin out dense clusters before generating the final 5cm resolution binary occupancy map.

3. Path Planning

For the global path-planning, we began with the bug2 path-planning algorithm, but due to the maze's complexity, settled on implementing the A* planner on our processed map. This was done due to the bug2 algorithm's lazy "Wall Following" nature being insufficient for generating the waypoints required to get from the start point to the goal. As such, while A* proved to be more computationally expensive, it was the optimal algorithm for the task. After the algorithm was run, the final path was resampled into waypoints used to provide smooth guidance targets for the robot's motion controller.

Results

Obstacle Avoidance

The potential field controller successfully navigated dynamic obstacles. Initial tests showed the robot getting "stuck" in local minima (corners), but increasing the repulsive field of view allowed the robot to "see" the corner earlier and turn away more smoothly.

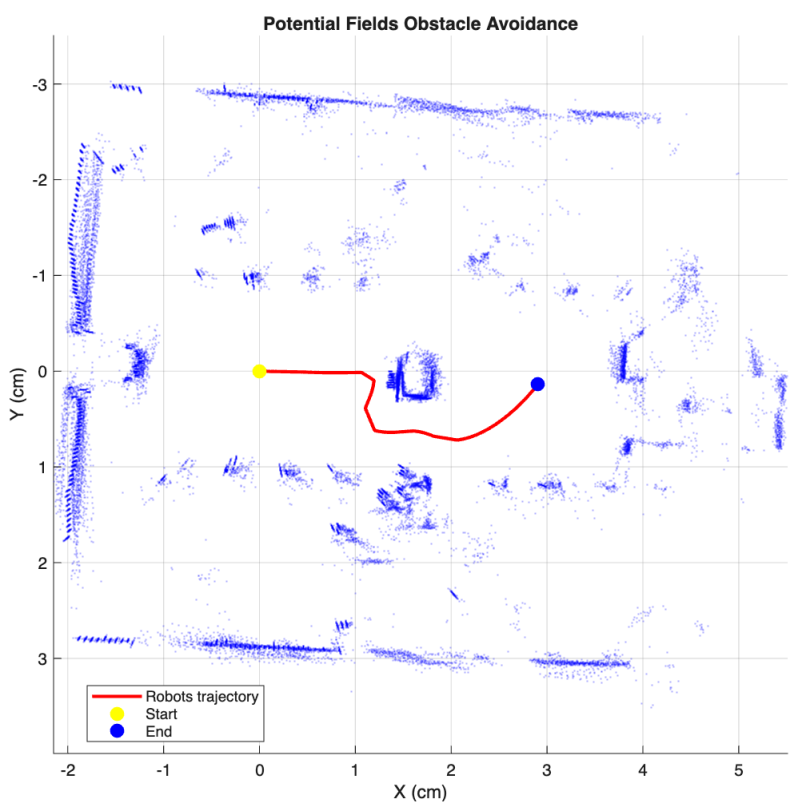

Fig. 1 Robot trajectory curving around obstacles using potential fields method for collision avoidance

Mapping & Cleaning

The raw mapping data was unusable for planning due to significant noise. After applying our custom cleaning pipeline (cropping and statistical outlier removal), we obtained a sharp, navigable occupancy grid. The walls became distinct red barriers in our heatmap, separated from the white corridors.

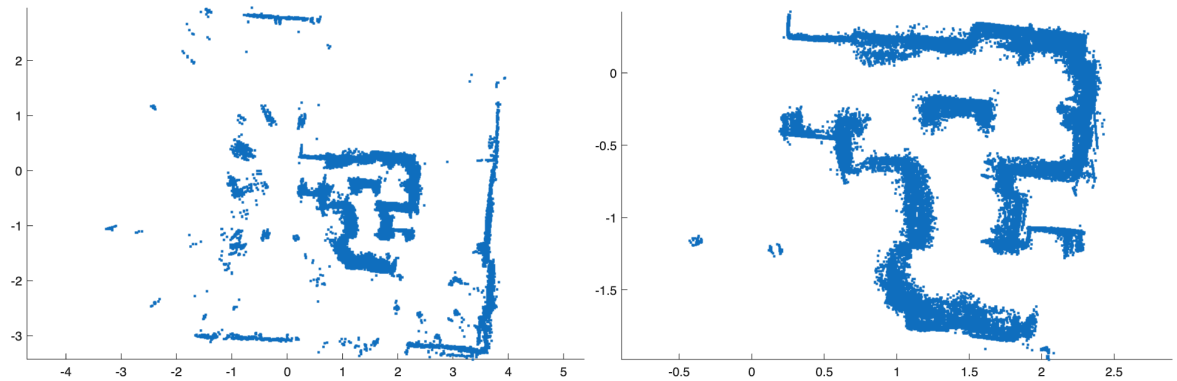

Fig. 2 Raw LIDAR point cloud before cleaning (left) with noise artifacts, and cleaned occupancy grid map (right) showing distinct wall barriers and navigation corridors

Path Planning

A* Performance: The A* planner found the shortest path through the white corridor, hugging the optimal line while respecting the set safety inflation. The heatmap visualization clearly showed the cost-to-go gradient spreading from the start to the goal.

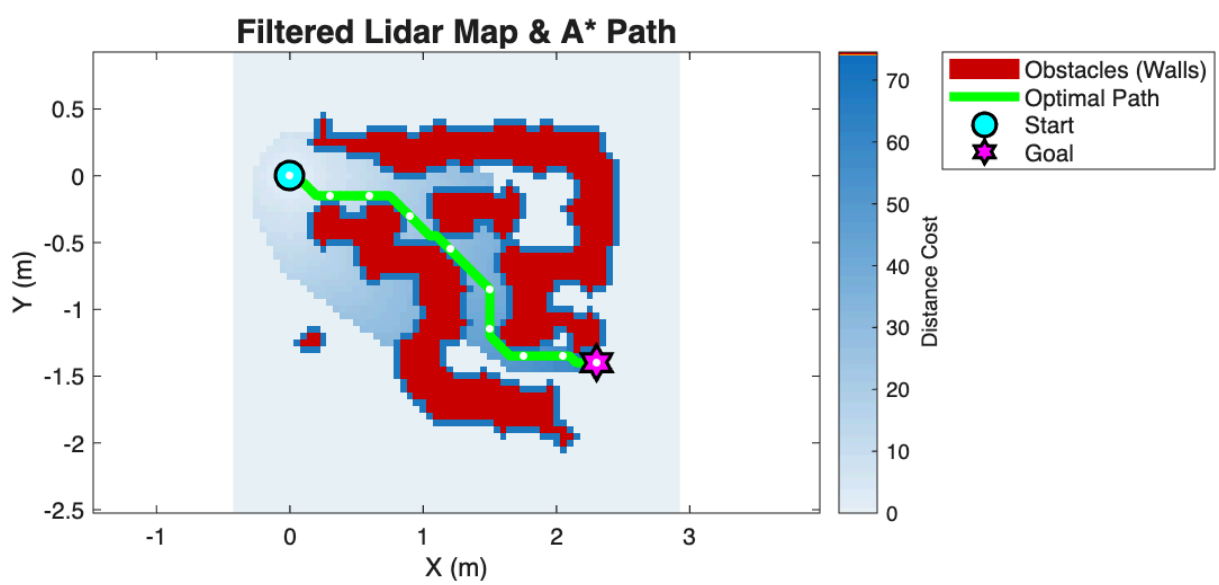

Fig. 3 Filtered occupancy grid map with red obstacles, green optimal A* path from start (cyan) to goal (purple star), and distance cost visualization

A* Trajectory Execution

We validated the A* planner by feeding the generated waypoints into the robot's motion controller in the actual environment. As seen in the trajectory plot below, the robot (red line) successfully tracked the path from the start to the goal point at [2.3, -1.4] in the graph, navigating the complex "S-turn" and maintaining a safe distance from the obstacle walls. The actual trajectory shows minor deviations due to controller lag, but the robot remained robustly within the safe corridor throughout the run.

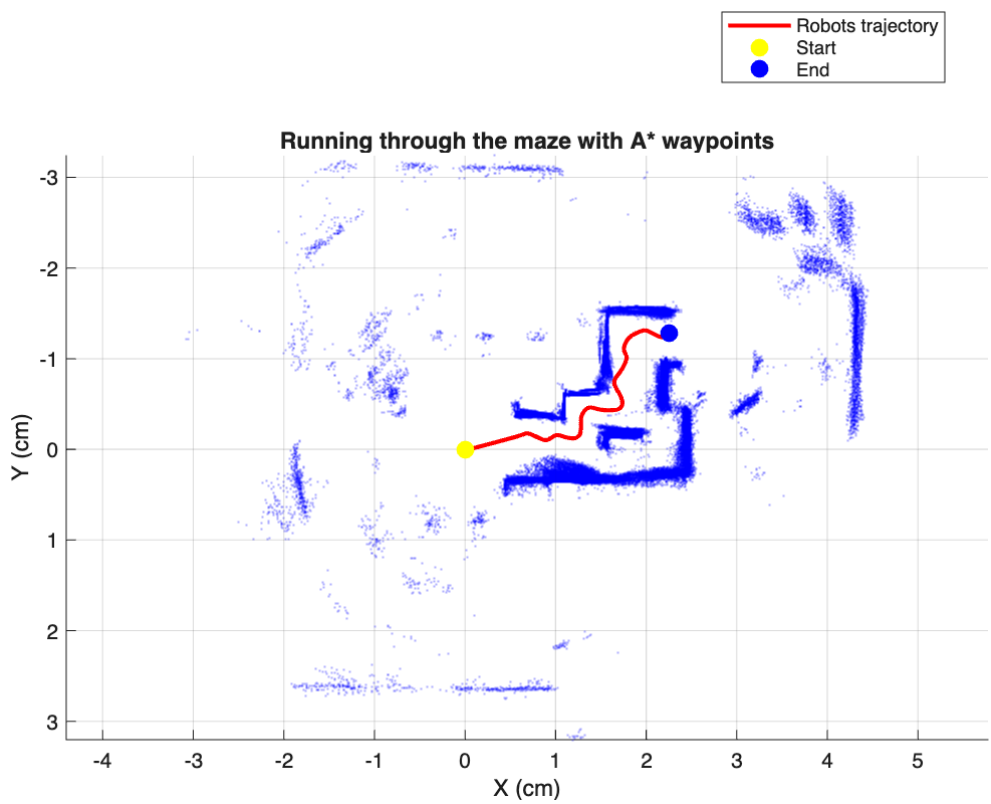

Fig. 4 Actual robot trajectory (red line) successfully navigating through the maze by following A* waypoints, overlaid on LIDAR point cloud (blue) showing real sensor data and wall obstacles

Discussion

Real Difficulties

The hardest part was data hygiene. A raw LIDAR scan looks clean to the human eye, but to an A* algorithm, a single stray pixel is an impassable brick wall. We spent significant time debugging "no path found" errors, only to realize tiny noise artifacts were blocking the corridor. Implementing the radius outlier rejection and morphological dilation was the turning point that made the planner reliable.

Another challenge was discretization. When the grid resolution was too coarse (>10cm), narrow doorways disappeared. When it was too fine (<2cm), the computation time spiked, and the path became jagged. We found 5cm to be the resolution for this environment.

The last challenge was tuning the values of the attractive and repulsive gains as well as the robot’s radius of influence to attract and repel objects in its range of field. Doing so required a proper study of the built maze to understand how the robot would react to passing through it, especially in areas where the path wasn’t as wide as it was used to.

Conclusion & Lessons Learned

In this lab, we learned that cleaning the data properly is more important than the kind of complex algorithm used to plan the path. While the simple Bug2 algorithm didn’t prove most optimal in this case, we are certain that it would have yielded better results than the sophisticated A* had we cleaned the map even more. We learned the hard truths of the messy reality of real-world sensor data, which highlight the need for preprocessing (cropping/filtering) and safety margins (inflation) in autonomous navigation.